BRAC University – Department of CSE

CSE 221: Algorithms | Mid Semester Exam – Spring 2026

Complete Model Answer Script

Question 1

Q1(a) – Why is Algorithm A still more efficient than Algorithm B? [2 marks]

Although $G_A(n) = 30n + 8$ produces larger values than $G_B(n) = n^2 + 1$ for small values of n (n = 3, 4, 5 in the table), the comparison must be made for **sufficiently large n**, which is the foundation of asymptotic (Big-O) analysis.

Key reasoning:

For n > 30, $G_B(n) = n^2 + 1$ exceeds $G_A(n) = 30n + 8$ and keeps growing much faster. Specifically, $G_A(n) = O(n)$ (linear) while $G_B(n) = O(n^2)$ (quadratic). A quadratic function eventually dominates any linear function, no matter how large the constant coefficient.

Crossover point:

Set $30n + 8 = n^2 + 1 \rightarrow n^2 - 30n - 7 = 0$. Solving: $n \approx 30.2$. So for n > 31, $G_B(n) > G_A(n)$ always.

Big-O notation ignores constants and small inputs. For large-scale inputs ($n \rightarrow \infty$), Algorithm A (O(n)) will always be faster than Algorithm B ($O(n^2)$). The initial region where $G_A > G_B$ is irrelevant in asymptotic analysis.

Q1(b) – Time Complexity of foo() in Big-O notation [3 marks]

```
int foo(int n) {
    int sum = 0;
    for(int i=1; i<=n; i*=4)          // Loop 1: i = 1,4,16,... → runs log■(n) times
    {
        for(int j=1; j<=n; j+=7)      // Loop 2: j increments by 7 → runs n/7 ≈ O(n) times
        {
            sum = sum + (i*j);
            for(int k=1; k<=n; k+=3)  // Loop 3: k increments by 3 → ALWAYS breaks at k=1
            {                          //          because k=1 is odd → k%2 != 0, break
                if (k%2==0) break;     // k starts at 1 (odd), so break fires immediately
            }                          // Loop 3 always executes exactly 1 iteration → O(1)
        }
    }
    return sum;
}
```

Analysis of each loop:

Loop	Variable	Step	Iterations	Complexity
Outer (i)	i = 1 → n	i *= 4	log■(n)	O(log n)
Middle (j)	j = 1 → n	j += 7	n/7	O(n)
Inner (k)	k = 1 → n	k += 3	1 (breaks instantly*)	O(1)

* The innermost loop: k starts at 1 (odd). The condition k%2==0 is false for k=1. So the break does NOT fire on k=1. The loop body runs once (sum updated), then k becomes 4; 4%2==0 → break. So the inner loop executes **exactly 2 iterations** → O(1).

Total complexity of foo(): T = O(log n) × O(n) × O(1) = **O(n log n)**

Q1(c) – Recurrence Relation for recur() and its Solution [2+3 marks]

Step 1 – Understanding recur():

```
int recur(A[], p, r) {
    int n = (r - p + 1);          // n = size of current subarray
    if (n <= 1) return 2;          // base case
    int b = recur(A[], p, p+(n/4));          // size = n/4 + 1 ≈ n/4
    int c = recur(A[], p+(n/4)+1, p+(n/2)); // size = n/4
    int d = recur(A[], p, p+(n/3));          // size = n/3 + 1 ≈ n/3
    int e = foo(n);                           // O(n log n)
    return b+c-d+e;
}
```

Step 2 – Subproblem sizes:

- b: recur on subarray of size ≈ n/4
- c: recur on subarray of size ≈ n/4
- d: recur on subarray of size ≈ n/3
- foo(n) costs O(n log n) [from Q1(b)]

Step 3 – Recurrence Relation:

T(n) = T(n/4) + T(n/4) + T(n/3) + O(n log n)

T(n) = 2·T(n/4) + T(n/3) + O(n log n)

T(1) = O(1) (base case)

Step 4 – Solution using Akra-Bazzi / Substitution:

We apply the **Akra-Bazzi method**. The recurrence has the form:

T(n) = a■·T(n/b■) + a■·T(n/b■) + a■·T(n/b■) + f(n)

where a■=2, b■=4; a■=1, b■=3 (treating 2T(n/4)+T(n/3)); f(n) = n log n

Find p such that: $a■/b■^{p} + a■/b■^{p} = 1$

→ $2/4^{p} + 1/3^{p} = 1$

At p=1: 2/4 + 1/3 = 0.5 + 0.333 = 0.833 < 1

At p=0.9: ~0.57 + 0.40 ≈ 0.97 < 1

At p=0.85: ~0.62 + 0.42 ≈ 1.04 > 1 → p ≈ 0.87 < 1

Since p < 1 and f(n) = n log n = n^{1}·log n grows faster than n^{p} (because 1 > p ≈ 0.87), the driving function **f(n) dominates**.

By Akra-Bazzi theorem: T(n) = Θ(n^{p} · (1 + ∫■■ $f(u)/u^{p+1}$ du))

= Θ(n^{p} · ∫ u·log(u)/u^{p+1} du) = Θ(n^{p} · ∫ log(u)/u^{p} du)

Since p < 1, this integral is dominated by n^{1}·log n.

∴ T(n) = Θ(n log n) — The recurrence is dominated by foo(n), giving O(n log n).

Question 2

List A (sorted): [2, 3, 7, 11, 13] List B (unsorted): [5, 1, 3, 3, 4]

Q2(a) – Sort List B using a Linear-Time Sorting Algorithm [3 marks]

Since all values are small non-negative integers, we use **Counting Sort**, which runs in O(n + k) time where n = number of elements and k = range of values.

Input: List B = [5, 1, 3, 3, 4] (n=5, values 1–5, k=5)

Step 1 – Build Count Array C[0..5] (indices 0 to max value 5):

Index	0	1	2	3	4	5
Count	0	1	0	2	1	1

→ 1 appears 1×, 3 appears 2×, 4 appears 1×, 5 appears 1×

Step 2 – Cumulative count (prefix sum):

Index	0	1	2	3	4	5
Cumul.	0	1	1	3	4	5

Step 3 – Build output array (scan B right to left for stability):

- B[4]=4 → place at output[C[4]-1]=output[3], C[4]-- → output=[_,_,_,4,_]
- B[3]=3 → place at output[C[3]-1]=output[2], C[3]-- → output=[_,_,3,4,_]
- B[2]=3 → place at output[C[3]-1]=output[1], C[3]-- → output=[_,3,3,4,_]
- B[1]=1 → place at output[C[1]-1]=output[0], C[1]-- → output=[1,3,3,4,_]
- B[0]=5 → place at output[C[5]-1]=output[4], C[5]-- → output=[1,3,3,4,5]

Result – Sorted List B = [1, 3, 3, 4, 5] Time: O(n+k) = O(n) ✓

Q2(b) – Merge Sorted Lists A and B into Sorted List C [3 marks]

List A = [2, 3, 7, 11, 13] Sorted B = [1, 3, 3, 4, 5]

Use the standard **two-pointer merge** (as in Merge Sort's merge step), O(n+m):

Step	i (ptr A)	j (ptr B)	Comparison	Output so far
1	A[0]=2	B[0]=1	1 < 2 → take B[0]	[1]
2	A[0]=2	B[1]=3	2 < 3 → take A[0]	[1,2]
3	A[1]=3	B[1]=3	3 = 3 → take A[1]	[1,2,3]
4	A[2]=7	B[1]=3	3 < 7 → take B[1]	[1,2,3,3]
5	A[2]=7	B[2]=3	3 < 7 → take B[2]	[1,2,3,3,3]
6	A[2]=7	B[3]=4	4 < 7 → take B[3]	[1,2,3,3,3,4]
7	A[2]=7	B[4]=5	5 < 7 → take B[4]	[1,2,3,3,3,4,5]
8	A[2]=7	—	B exhausted, copy rest of A	[1,2,3,3,3,4,5,7,11,13]

Sorted List C = [1, 2, 3, 3, 3, 4, 5, 7, 11, 13] Time: O(n+m) = O(n) ✓

Q2(c) – Binary Search for Pivot-Avoidance Indices in Rotated Array [3 marks]

List C = [1, 2, 3, 3, 3, 4, 5, 7, 11, 13]

Rotate 3 positions LEFT: remove first 3 → [3, 3, 4, 5, 7, 11, 13], then append [1, 2, 3]

List D = [3, 3, 4, 5, 7, 11, 13, 1, 2, 3] (indices 0–9)

Pivot avoidance rule: We must NOT pick the **minimum** or **maximum** element as pivot in a rotated sorted array on the first partition, as these would leave one partition empty. In D, min = 1 (index 7) and max = 13 (index 6).

Algorithm – Find the rotation point (index of minimum) in O(log n):

This also indirectly finds the maximum (it is at index (min_index – 1 + n) % n).

```
// Binary Search to find indices of min and max in rotated sorted array D
// Returns: list of indices to AVOID as first pivot

function findAvoidIndices(D[], n):
    // Step 1: find index of minimum element (rotation point)
    lo = 0
    hi = n - 1

    while (lo < hi):
        mid = (lo + hi) / 2
        if D[mid] > D[hi]:
            lo = mid + 1   // min is in right half
        else:
            hi = mid       // min is in left half or at mid

    min_idx = lo
    max_idx = (lo - 1 + n) % n   // element just before min is the maximum

    // Return indices to avoid
    return [min_idx, max_idx]

// ----- Simulation on D = [3,3,4,5,7,11,13,1,2,3], n=10 -----
// lo=0, hi=9:  mid=4, D[4]=7, D[9]=3  → 7>3 → lo=5
// lo=5, hi=9:  mid=7, D[7]=1, D[9]=3  → 1<3 → hi=7
// lo=5, hi=7:  mid=6, D[6]=13,D[7]=1  → 13>1 → lo=7
// lo=7, hi=7:  loop ends
// min_idx = 7  (D[7]=1)
// max_idx = (7-1+10)%10 = 6  (D[6]=13)
// AVOID indices: {6, 7}  i.e., values 13 and 1
```

Time Complexity: O(log■ n) — one binary search. Avoid pivots at indices 6 (value 13) and 7 (value 1).

Q2(d) – Issue with Counting Sort when IDs can be up to 1,000,000,000 [1 mark]

Counting Sort requires an auxiliary count array of size **k+1**, where k is the maximum possible value. If student IDs can now be any integer up to 1,000,000,000 (10^9), the count array would need **1 billion entries**.

Issues:

1. **Space infeasibility:** Allocating an array of size 10^9 integers requires ~4 GB of memory — completely impractical.

2. **Time infeasibility:** Counting Sort's complexity is O(n + k). With k = 10^9 and only ~5 students, initialising the count array alone takes O(10^9) >> O(n), making it far slower than comparison-based sorts.

Fix: Use a comparison-based sort like Merge Sort O(n log n) or, if the IDs are sparse, coordinate-compress them first.

Question 3

Q3(a) – Demonstrate MaxGCDSubarray on A = {6, 10, 35, 15} [3 marks]

Array A = [6, 10, 35, 15], indices 0–3. The true maximum GCD subarray is {35} with GCD = 35 (single element). Let us trace the algorithm to see whether it finds this.

Recursive call tree:

```
MaxGCDSubarray(A, 0, 3):
mid = 1
LGS = MaxGCDSubarray(A, 0, 1):
mid = 0
LGS = MaxGCDSubarray(A, 0, 0) → return A[0] = 6
RGS = MaxGCDSubarray(A, 1, 1) → return A[1] = 10
CGS = CrossGCD(A, 0, 0, 1):
left side (i from 0 down to 0): g=GCD(0,6)=6 → leftGCD=6
right side (i from 1 to 1): g=GCD(0,10)=10 → rightGCD=10
return GCD(6,10) = 2
LGS = max(6, 10, 2) = 10

RGS = MaxGCDSubarray(A, 2, 3):
mid = 2
LGS = MaxGCDSubarray(A, 2, 2) → return A[2] = 35
RGS = MaxGCDSubarray(A, 3, 3) → return A[3] = 15
CGS = CrossGCD(A, 2, 2, 3):
left side (i from 2 down to 2): g=GCD(0,35)=35 → leftGCD=35
right side (i from 3 to 3): g=GCD(0,15)=15 → rightGCD=15
return GCD(35,15) = 5
RGS = max(35, 15, 5) = 35

CGS = CrossGCD(A, 0, 1, 3):
left side (i from 1 down to 0):
i=1: g=GCD(0,10)=10, leftGCD=10
i=0: g=GCD(10,6)=2, leftGCD stays 10
right side (i from 2 to 3):
i=2: g=GCD(0,35)=35, rightGCD=35
i=3: g=GCD(35,15)=5, rightGCD stays 35
return GCD(10, 35) = 5
```

```
MaxGCDSubarray(A,0,3) = max(LGS=10, RGS=35, CGS=5) = 35 ✓
```

Conclusion:

The algorithm correctly returns **35**, which is the true Max-GCD. It works properly for this input because the maximum GCD element (35) is a single-element subarray that appears as a leaf of the recursion. The CrossGCD step correctly computes GCD of cross-boundary subarrays but never exceeds a single element's value when GCDs reduce quickly. ✓ **The algorithm works correctly on A = {6, 10, 35, 15}.**

> **When does it fail?** The algorithm can fail when the optimal subarray crosses the midpoint but CrossGCD misses it. For example if the best cross-subarray had a higher GCD achievable only by NOT taking the element closest to mid (since CrossGCD greedily expands from mid outward, tracking only the best GCD seen so far, which is always non-increasing as we expand). However for this particular input it gives the right answer.

Q3(b) – Recurrence Relation with Fast_GCD [2 marks]

With the standard Euclidean GCD (O(log N) per call), CrossGCD scans O(n) elements and calls GCD O(n) times, costing O(n log N) where N is array size. Now with **Fast_GCD** costing O(√N) per call:

CrossGCD cost with Fast_GCD:

> CrossGCD scans O(n) elements. Each call to Fast_GCD(g, A[i]) now costs O(√N), where N is the size of the *original* array (since 1 ≤ A[i] ≤ N). Therefore CrossGCD costs O(n · √N).

At the top level (n = N), the cost of CrossGCD is O(N · √N) = $O(N^{3/2})$.

Recurrence Relation:

T(n) = 2 · T(n/2) + O(n · √N)

> T(1) = O(1)

Here n is the current subproblem size (shrinks with recursion) and N is the original array size (constant, since A[i] ≤ N is always the original constraint). The driving function f(n) = O(n√N) is linear in n with constant √N factor.

By Master Theorem: a=2, b=2, f(n)=Θ(n√N). $\log_b a = \log■2 = 1$. $f(n) = \Theta(n^1 \cdot \sqrt{N}) = \Theta(n) \times \sqrt{N}$. This is Case 2 ($f(n) = \Theta(n^{\log_b\ a} \cdot \sqrt{N})$), giving: **$T(N) = O(N \cdot \sqrt{N} \cdot \log N) = O(N^{3/2} \log N)$**

Question 4

Graph (from Figure 1): Nodes: {A, B, C, D, E, F, G, H, X}

Edges (undirected): C–A, C–F, A–X, A–D, X–B, F–D, F–H, D–B, D–G, B–E, H–G, G–E

Q4(a) – BFS from X to H: Minimum Time Steps [4 marks]

Signal transmission = BFS (each time unit = one BFS level). Alphabetical tie-breaking. Start: X.

Adjacency list (alphabetical order per node):

```
A: C, D, X
B: D, E, X
C: A, F
```

```
D: A, B, F, G
E: B, G
F: C, D, H
G: D, E, H
H: F, G
X: A, B
```

BFS Simulation:

Time	Queue (front→back)	Visited	Dequeued	Neighbours Added
0 (init)	[X]	{X}	—	—
t=1	[A, B]	{X,A,B}	X	A, B (alphabetical)
t=2	[C, D, E]	{X,A,B,C,D,E}	A then B	A→{C,D}; B→{E}
t=3	[F, G, H]	{X,A,B,C,D,E,F,G,H}	C,D,E	C→{F}; D→{G}; E→{H? no}→G already; F added from C
				D→{F already}, {G new}; E→{G already}
H reached!	H discovered at t=3	✓		

Detailed BFS step-by-step:

- Queue=[X], Visited={X}
- Dequeue X → Neighbours: A, B (alphabetical). Enqueue A, B. Distance[A]=1, Distance[B]=1.
- Queue=[A, B]
- Dequeue A → Neighbours: C, D, X. X visited. Enqueue C, D. Distance[C]=2, Distance[D]=2.
- Dequeue B → Neighbours: D, E, X. D,X visited. Enqueue E. Distance[E]=2.
- Queue=[C, D, E]
- Dequeue C → Neighbours: A, F. A visited. Enqueue F. Distance[F]=3.
- Dequeue D → Neighbours: A, B, F, G. A,B visited; F just enqueued. Enqueue G. Distance[G]=3.
- Dequeue E → Neighbours: B, G. Both visited or enqueued.
- Queue=[F, G]
- Dequeue F → Neighbours: C, D, H. C,D visited. Enqueue H. Distance[H] = 3+1 = 4.
- Wait — H is a neighbour of F. F is at distance 3, so H is at distance 4.
- Also check: G → Neighbours: D, E, H. Dequeue G, H found again at distance 4.

Shortest path: X → A → C → F → H or X → A → D → G → H (both length 4)

Minimum time steps to reach H from X = 4

Queue / Visited state summary:

Time Step	Nodes Receiving Signal	Queue After Processing
0	X (source)	[X]
1	A, B	[A, B]
2	C, D, E	[C, D, E]
3	F, G	[F, G]
4	H ← reached!	H dequeued

Q4(b) – Frequency Assignment: 2-Coloring / Bipartiteness Check [3 marks]

Two frequencies can be assigned such that no two adjacent stations share the same frequency **if and only if the graph is bipartite**. We can check this using BFS-coloring.

BFS 2-Coloring Simulation starting from X with freq_A:

Station(s)	Assigned Frequency	Reason
X	freq_A	—
A, B	freq_B	neighbours of X
C, D, E	freq_A	neighbours of A and B
F, G	freq_B	neighbours of C, D, E
H	freq_A	neighbour of F and G

Verification – check all edges for conflict:

C–A: freq_A – freq_B ✓ C–F: freq_A – freq_B ✓
A–X: freq_B – freq_A ✓ A–D: freq_B – freq_A ✓
X–B: freq_A – freq_B ✓ F–D: freq_B – freq_A ✓
F–H: freq_B – freq_A ✓ D–B: freq_A – freq_B ✓
D–G: freq_A – freq_B ✓ B–E: freq_B – freq_A ✓
H–G: freq_A – freq_B ✓ G–E: freq_B – freq_A ✓

No conflicts found! ∴ The graph IS bipartite. Frequency assignment IS possible.

freq_A group: X, C, D, E, H

freq_B group: A, B, F, G

Q4(c) – Modified BFS with Locked Stations and Keys [3 marks]

Some stations are **locked**: you cannot visit them unless you already hold their key. A **rescuer station**, when visited, gives you the key to one specific locked station. We start with no keys (only station X is available).

Modification Strategy:

The core idea is to augment the standard BFS with a **key-tracking mechanism**. When a rescuer station is visited, its key is added to our key-set, which may unlock a previously-blocked locked station so it can be enqueued.

Modified BFS Algorithm:

```
Modified-BFS(graph, start, end, locked_map, rescuer_map):
// locked_map: locked_station → required_key_name
// rescuer_map: rescuer_station → key_name_it_provides

    keys = {}                    // set of keys we currently hold
    visited = {start}
    queue = [(start, dist=0)]
    pending_locked = {}          // key_name → list of locked stations waiting for this key

    while queue is not empty:
        (u, d) = dequeue(queue)

        if u == end:
            return d             // minimum time steps found
        // If u is a rescuer station, collect the key it provides
```

```
        if u in rescuer_map:
            new_key = rescuer_map[u]
            keys.add(new_key)

            // Check if any pending locked stations are now unlocked
            if new_key in pending_locked:
                for locked_v in pending_locked[new_key]:
                    if locked_v not in visited:
                        visited.add(locked_v)
                        enqueue(queue, (locked_v, d+1))
                delete pending_locked[new_key]

        // Explore neighbours of u (in alphabetical order for tie-breaking)
        for each neighbour v of u (sorted alphabetically):
            if v not in visited:
                if v in locked_map:
                    required_key = locked_map[v]
                    if required_key in keys:
                        // We have the key → can visit
                        visited.add(v)
                        enqueue(queue, (v, d+1))
                    else:
                        // Cannot visit yet → add to pending list
                        pending_locked[required_key].append(v)
                else:
                    // Not locked → visit normally
                    visited.add(v)
                    enqueue(queue, (v, d+1))

    return -1  // H not reachable
```

Key points of the modification:

- **Key Set:** Maintain a set 'keys' of all keys collected so far.
- **Pending Map:** When a locked station v is encountered without its key, store v in a pending map keyed by the required key name.
- **Key Collection:** When a rescuer station is dequeued, add its key to 'keys', then immediately release all pending locked stations that needed that key — enqueue them at current distance + 1.
- **Correctness:** Since the problem guarantees no infinite loops in the unlocking chain, the BFS terminates. The pending mechanism ensures the minimum-distance property is preserved.
- **Complexity:** O(V + E) same as standard BFS — each station is visited at most once.